

CSE312 HW1
PART-B REPORT

MUHAMMET FURKAN YILGIN
22010400496

1.Lifecycles of PART-B

2.1 - Lifecycle 1

Here is the implementation of lifecycle 1 with the comments and the outputs of the lifecycle 1 below.

Implementation :

```
void lifecycle1(GlobalDescriptorTable *gdt, TaskManager *tm){
    // Loading one of the selected program 10 times.
    int program_num = 4; // number of program types.
    void (*prog[program_num]) (void) = {LongRunningProgramTesting ,LinearSearchTesting, BinarySearchTesting, CollatzSequenceTesting};
    void (*selection)(void) = prog[rand(0, program_num)]; // selecting one of them.
    Task task;
    // loading the selected program 10 times.
    for (int i = 0; i < 10; ++i) {
        task.InitTask(gdt, selection, 1, 1, Normal);
        tm->AddTask(&task);
    }
}
```

Outputs :

```
Welcome to FurkanOS
LIFECYCLE 1 OF PART B
--- BINARY SEARCH ---
OUR ARRAY IS -> 10 20 80 30 60 50 110 100 130 170
SORTING THE ARRAY...
SORTED ->10 20 30 50 60 80 100 110 130 170
OUR TARGET IS -> 50
Result of binary search for target '50' is -> 3
--- EXIT SYSCALL IS CALLED ---
--- BINARY SEARCH ---
OUR ARRAY IS -> 10 20 80 30 60 50 110 100 130 170
SORTING THE ARRAY...
SORTED ->10 20 30 50 60 80 100 110 130 170
OUR TARGET IS -> 80
Result of binary search for target '80' is -> 5
--- BINARY SEARCH ---
OUR ARRAY IS -> 10 20 80 30 60 50 110 100 130 170
SORTING THE ARRAY...
SORTED ->10 20 30 50 60 80 100 110 130 170
OUR TARGET IS -> 110
```

```

SORTED ->10 20 30 50 60 80 100 110 130 170
OUR TARGET IS -> 110
Result of binary search for target '110' is -> 7
--- BINARY SEARCH ---
OUR ARRAY IS -> 10 20 80 30 60 50 110 100 130 170
SORTING THE ARRAY...
SORTED ->10 20 30 50 60 80 100 110 130 170
OUR TARGET IS -> 50
Result of binary search for target '50' is -> 3
--- EXIT SYSCALL IS CALLED ---
--- BINARY SEARCH ---
OUR ARRAY IS -> 10 20 80 30 60 50 110 100 130 170
SORTING THE ARRAY...
SORTED ->10 20 30 50 60 80 100 110 130 170
OUR TARGET IS -> 10
Result of binary search for target '10' is -> 0
--- BINARY SEARCH ---
OUR ARRAY IS -> 10 20 80 30 60 50 110 100 130 170
SORTING THE ARRAY...
SORTED ->10 20 30 50 60 80 100 110 130 170
OUR TARGET IS -> 170

```

```

SORTING THE ARRAY...
SORTED ->10 20 30 50 60 80 100 110 130 170
OUR TARGET IS -> 170KEYBOARD 0x0E
Result of binary search for target '170' is -> 9
--- BINARY SEARCH ---
OUR ARRAY IS -> 10 20 80 30 60 50 110 100 130 170
SORTING THE ARRAY...
SORTED ->10 20 30 50 60 80 100 110 130 170
OUR TARGET IS -> 130
Result of binary search for target '130' is -> 8
--- EXIT SYSCALL IS CALLED ---
--- BINARY SEARCH ---
OUR ARRAY IS -> 10 20 80 30 60 50 110 100 130 170
SORTING THE ARRAY...
SORTED ->10 20 30 50 60 80 100 110 130 170
OUR TARGET IS -> 10
Result of binary search for target '10' is -> 0
--- EXIT SYSCALL IS CALLED ---
--- EXIT SYSCALL IS CALLED ---
--- EXIT SYSCALL IS CALLED ---
--- EXIT SYSCALL IS CALLED ---
--- EXIT SYSCALL IS CALLED ---

```

2.2 - Lifecycle 2

Here is the implementation of lifecycle 2 with the comments and the outputs of the lifecycle 2 below.

Implementation :

```
void lifecycle2(GlobalDescriptorTable *gdt, TaskManager *tm){
    // Loading 2 of the selected program 3 times each.
    int program_num = 4;
    // to select between the 4 of the programs.
    void (*prog[program_num])(void) = {LongRunningProgramTesting, LinearSearchTesting, BinarySearchTesting, CollatzSequenceTesting};

    int selection1 = rand(0, program_num); // selecting the 1st program.
    int selection2 = rand(0, program_num); // selecting the 2nd program.
    while (selection2 == selection1) // if selection2 is the same number as selection1
        selection2 = rand(0, program_num); // selecting a new number until it is a different number than selection1.
    Task task;
    for (int i = 0; i < 3; ++i) { // loading the 2 of the selected programs 3 times each.
        task.InitTask(gdt, prog[selection1], 1, 1, Normal); // 1st one.
        tm->AddTask(&task);
        task.InitTask(gdt, prog[selection2], 1, 1, Normal); // 2nd one.
        tm->AddTask(&task);
    }
}
```

Outputs :

```
Welcome to FurkanOS
LIFECYCLE 2 OF PART B
--- LONG RUNNING PROGRAM ---
Result of 100 is -> 24502500
--- EXIT SYSCALL IS CALLED ---
--- LINEAR SEARCH ---
OUR ARRAY IS -> 10 20 80 30 60 50 110 100 130 170
VALUE TO BE FOUND IS -> 80
LINEAR-SEARCH(80): 2
--- LONG RUNNING PROGRAM ---
Result of 100 is -> 24502500
--- EXIT SYSCALL IS CALLED ---
--- LINEAR SEARCH ---
OUR ARRAY IS -> 10 20 80 30 60 50 110 100 130 170
VALUE TO BE FOUND IS -> 90
LINEAR-SEARCH(90): -1
--- LONG RUNNING PROGRAM ---
Result of 100 is -> 24502500
--- EXIT SYSCALL IS CALLED ---
--- LINEAR SEARCH ---
OUR ARRAY IS -> 10 20 80 30 60 50 110 100 130 170
VALUE TO BE FOUND IS -> 130
```

It gave the output of :

LINEAR-SEARCH(130): 8 and

-- EXIT SYSCALL IS CALLED 3 times , but I couldn't catch it since it went to the other screen but it is clearly seen that it chooses 2 different programs and runs each of them 3 times.

2. Round-Robin Scheduling

I would like to show how the Round-Robin Scheduling is implemented using the photo below with the comments.

Implementation :

```
CPUSState* TaskManager::Schedule(CPUSState* cpustate){
    if (Task_counter <= 0) // if there is no tasks to schedule which means no context switch will occur because there is nothing to switch to.
        // Therefore the current cpu state will be returned.
        return cpustate;

    if (currentTask >= 0) // If there is a current task being executed save its CPU state.
        // The CPU state includes registers, stack pointer, program counter, etc.
        tasks[currentTask].cpustate = cpustate; // The current CPU state (cpustate) is saved to the current task's entry in the task table.

    ControlTask();

    // printing the process table
    // it is commented because when it is executed the outputs can not be seen clearly for lifecycle.
    /*if (Active_Task_counter > 1)
        GetTaskStatus();*/

    return tasks[currentTask].cpustate;
    // After selecting the next task to run (via ControlTask()), the function returns the CPU state of this newly selected task.
}
```

3. Task Addition and Control

```
void TaskManager::CopyTask(Task* source, Task* destination) {
    // Copy basic task attributes from source to destination
    destination->state = source->state;
    destination->pid = source->pid;
    destination->ppid = source->ppid;
    destination->priority = source->priority;
    destination->waitparent = source->waitparent;

    // Copy the entire stack from source to destination
    for (int i = 0; i < Task::STACK_SIZE; ++i) {
        destination->stack[i] = source->stack[i];
    }

    // Copy the CPU state from source to destination
    *(destination->cpustate) = *(source->cpustate);
}

bool TaskManager::AddTask(Task* task) {
    // Check if the task limit has been reached
    if (Task_counter >= MAX_TASKS) {
        return false; // Cannot add more tasks if the limit is reached
    }

    // Assign the next available PID to the new task
    task->pid = next_pid;

    // Copy the task into the tasks array at the position indicated by Task_counter
    CopyTask(task, &tasks[Task_counter]);

    // Update the task manager's counters and PID
    IncrementPIDS_TASKS();

    return true; // Task added successfully
}
```

```
void TaskManager::IncrementPIDS_TASKS() {
    // Increment the PID for the next task to be added
    ++next_pid;

    // Increment the total number of tasks
    ++Task_counter;

    // Increment the number of active tasks
    ++Active_Task_counter;
}
```



```

void TaskManager::ControlTask() {
    // If the current task is running, mark it as ready to be scheduled again
    if (tasks[currentTask].state == Running)
        tasks[currentTask].state = Ready;

    do {
        // Move to the next task in the list
        // Apply Round Robin scheduling: increment currentTask
        // If currentTask exceeds the number of tasks, wrap around using modulo
        if (++currentTask >= Task_counter)
            currentTask %= Task_counter;

        // Loop until a task in the ready state is found
        // This ensures we skip any tasks that are not ready to run (blocked or terminated)
    } while (tasks[currentTask].state != Ready);

    // Once a ready task is found, mark it as running
    tasks[currentTask].state = Running;
}

```

```

void Task::InitTask(GlobalDescriptorTable *gdt, void entrypoint(), common::uint32_t pid, common::uint32_t ppid, TaskPriority priority) {
    // Initialize task parameters
    this->ppid = ppid; // Set parent process ID
    this->pid = pid; // Set process ID
    this->priority = priority; // Set task priority
    state = Ready; // Set initial state to Ready
    waitparent = false; // Initialize waitparent flag to false

    // Initialize CPU state pointer to the top of the stack minus the size of CPUState
    cpustate = (CPUState*)(stack + STACK_SIZE - sizeof(CPUState));

    // Initialize general-purpose registers to 0
    cpustate->eax = 0;
    cpustate->ebx = 0;
    cpustate->ecx = 0;
    cpustate->edx = 0;

    // Initialize additional general-purpose registers to 0
    cpustate->esi = 0;
    cpustate->edi = 0;
    cpustate->ebp = 0;

    // Set the instruction pointer to the entry point of the task
    cpustate->eip = (uint32_t)entrypoint;

    // Set the code segment selector using the GDT
    cpustate->cs = gdt->CodeSegmentSelector();

    // Set the initial flags register value
    cpustate->eflags = 0x202; // Enable interrupts (bit 9) and set reserved bits
}

```

Unfortunately , even if I tried to implement lifecycle 3 and dynamic priority strategy and mouse, keyboard interrupts(PART C) I failed to satisfy the given conditions.

Overall , I implemented the given algorithms of linear and binary search , collatz sequence and long running programs.I succeeded in using these programs in lifecycle of part A and lifecycle1 and lifecycle2 of part B.